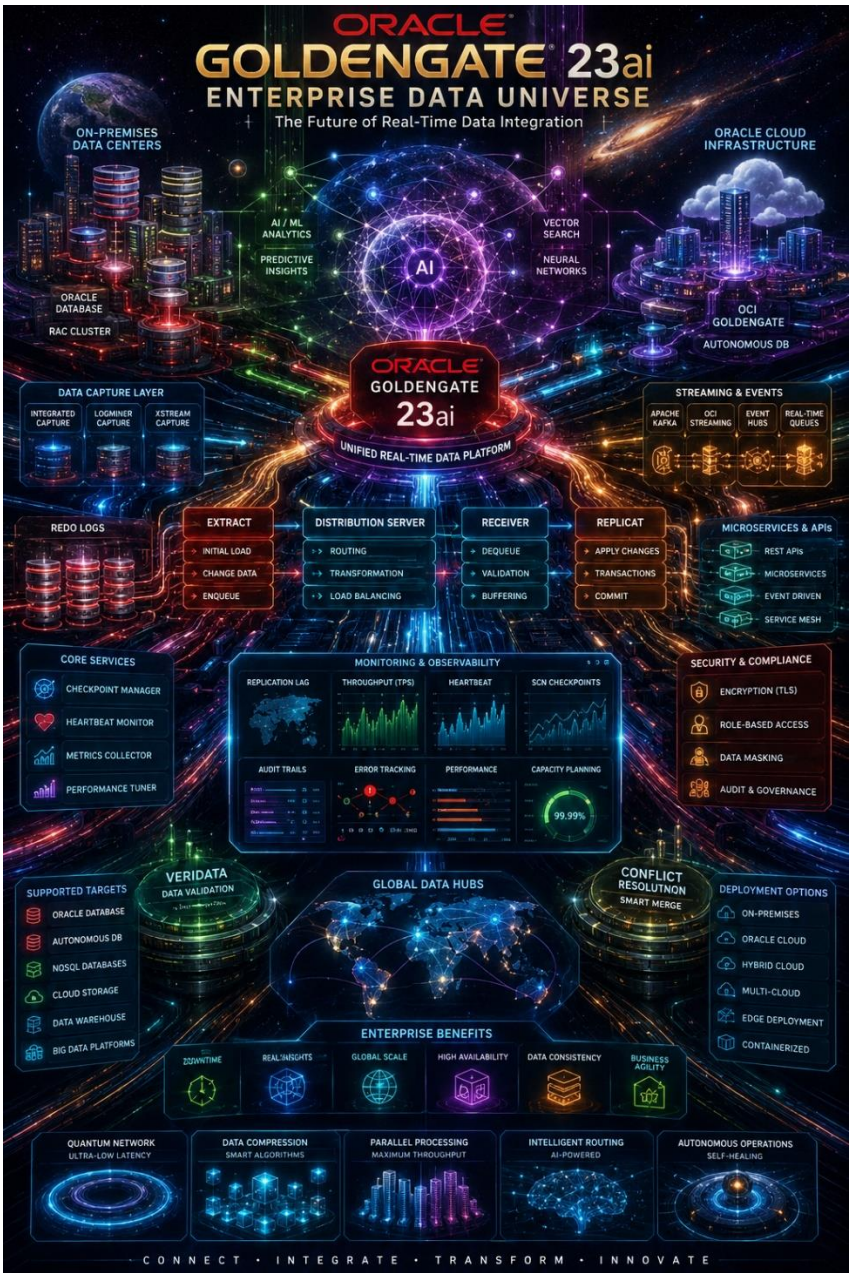
ORACLE GOLDENGATE

COMPLETE ENTERPRISE REFERENCE BOOK

Architecture | Internals | Administration | Performance | Troubleshooting



Chapter 1: The Complete Evolution of Oracle GoldenGate

Oracle GoldenGate did not start its life inside Oracle. The product was originally built by a company simply called GoldenGate Software, founded in the early 1990s, with the core idea of capturing changes from a database transaction log and moving those changes to another database in near real time. The original use case was fairly narrow: banks and telecom companies wanted a way to keep a reporting database in sync with a production database without putting extra load on production through triggers or batch jobs. Reading the redo stream directly, instead of scanning tables, was the breakthrough that made this practical.

Oracle acquired GoldenGate Software in 2009, mainly to strengthen its data integration and replication portfolio, which until then relied heavily on Oracle Streams. Streams was already showing its age — it was complex to manage, difficult to troubleshoot, and did not scale well for high volume OLTP systems. GoldenGate, being log-based, lightweight, and heterogeneous by design, was a far better long term direction. Streams was eventually deprecated and GoldenGate became the strategic replication technology for Oracle Database, and later for practically every major database engine on the market.

1.1 Major Release Timeline and What Changed

Understanding how GoldenGate evolved version by version is useful because production environments rarely run the newest release. A GoldenGate administrator frequently supports 12c, 19c, and 21c deployments side by side, and knowing what changed helps in planning upgrades and explaining behaviour differences to application teams.

Release Line	Approx. Timeframe	Key Change
GoldenGate 10/11 (Classic)	2009 - 2013	Classic Capture and Classic Delivery only, GGSCI as the sole interface, trail files in the original binary format.
GoldenGate 12c (12.1 / 12.2 / 12.3)	2014 - 2017	Integrated Capture and Integrated Replicat introduced, Coordinated Replicat added for parallel apply, Credential Store replaced clear text passwords.
GoldenGate 19c	2019 onward	Microservices Architecture matured as an alternative to Classic, Parallel Replicat introduced, long term support release widely adopted in production.
GoldenGate 21c	2021 onward	Microservices positioned as the default deployment model, further Parallel Replicat improvements, tighter integration with

Release Line	Approx. Timeframe	Key Change
		Autonomous Database and OCI GoldenGate.
GoldenGate 23ai	2023 - 2026	AI related capabilities added around JSON, vector data replication and Autonomous integration, Classic Architecture placed in maintenance mode for many platforms, Microservices REST and Admin Client treated as the primary management path.

WHY THIS MATTERS

A parameter or command that works perfectly on a 12c Classic install may behave differently, or may not exist at all, on a 23ai Microservices deployment. Always confirm the release and the deployment model — Classic or Microservices — before applying any configuration seen in a document, forum post, or an old runbook.

1.2 Architecture Evolution — From Classic to Microservices

The original architecture, now referred to as Classic Architecture, is built around GGSCI (GoldenGate Software Command Interpreter), flat parameter files, and OS-level processes managed by a Manager process. It is simple to understand and still works reliably, which is why so many production sites continue running it. However, it has weaknesses: no built-in REST interface, weaker security model, manual process management on each host, and no unified way to see all deployments across a data center from one screen.

Microservices Architecture was introduced to solve exactly these gaps. Instead of a single Manager process babysitting Extract and Replicat, there is a Service Manager that starts and monitors a set of independent microservices — Administration Server, Distribution Server, Receiver Server, Performance Metrics Server, and optionally a Non-Secure or Secure listener depending on TLS configuration. Each of these exposes REST endpoints, so automation, monitoring dashboards, and OCI GoldenGate itself can talk to GoldenGate without ever opening GGSCI.

1.2.1 Admin Client Replacing GGSCI

In Microservices deployments, the primary command line tool is Admin Client (adminclient), not GGSCI. Admin Client connects to the Administration Server over REST, and the command syntax is intentionally close to GGSCI so that experienced administrators do not have to relearn everything from scratch. That said, Admin Client is not a full replacement in every corner case — some legacy Classic-only parameters and DBLOGIN style workflows do not carry over cleanly, and this catches out administrators who assume the two tools are interchangeable.

```
-- Classic Architecture, connecting with GGSCI
GGSCI> DBLOGIN USERIDALIAS ggadmin
GGSCI> INFO EXTRACT EXT1
```

```
-- Microservices Architecture, connecting with Admin Client
adminclient> CONNECT https://ggnode1:9010 AS oggadmin PASSWORD ******
adminclient> INFO EXTRACT EXT1, DOMAIN admin
```

1.3 Capture and Apply Evolution

Capture started as Classic Capture, which reads the online and archived redo logs directly from the operating system. This works fine for most workloads but places a dependency on Extract having OS level access to redo, and it does not understand certain newer data types cleanly. Integrated Capture moved the log mining work inside the database kernel itself, using the same infrastructure as LogMiner. This gave Oracle tighter control over consistency, better handling of compressed tables, and simpler support for RAC environments because Extract no longer needed to merge redo from multiple threads manually.

On the apply side, Classic Replicat applies transactions serially, one at a time, which is easy to reason about but slow for high volume targets. Integrated Replicat pushes the apply work into the database using an inbound server, letting Oracle decide how to parallelize conflicting and non-conflicting transactions safely. Coordinated Replicat was Oracle's earlier attempt at manual parallelism, where the administrator defines how many threads to run and which rows go to which thread using a hash on key columns. Parallel Replicat, introduced later, automates this decision-making, dynamically building a dependency graph of transactions and applying independent transactions concurrently without the administrator having to manually size threads.

Replicat Type	Parallelism Model	Typical Use Case
Classic Replicat	Single threaded, serial apply	Small volume, simple mapping, easy troubleshooting
Coordinated Replicat	Manually defined threads with hash-based row distribution	Medium to high volume where the DBA can predict a good split key
Integrated Replicat	Database-managed parallelism through inbound server	Oracle to Oracle replication where the target can absorb parallel apply
Parallel Replicat	Automatic dependency-based parallelism, no manual thread sizing	High volume OLTP targets, both Oracle and select non-Oracle targets

1.4 Big Data, Kafka, and Cloud Direction

As data platforms diversified, Oracle extended GoldenGate beyond database-to-database replication. GoldenGate for Big Data added handlers for Kafka, HDFS, Elasticsearch, and similar targets, letting a single Extract feed both a traditional Oracle standby-like target and a streaming platform at the same time. Kafka integration in particular became a common pattern for feeding microservices architectures and real time analytics platforms, since a trail file can be converted into a stream of change events without writing custom capture code.

On the cloud side, OCI GoldenGate packages the Microservices Architecture as a managed service, removing the need to patch, size, and babysit the Service Manager tier. OCI GoldenGate deployments still use the same trail file format and largely the same parameter syntax, so skills built on-premises transfer directly to the cloud offering. GoldenGate Hub, a customer managed on-premises equivalent, groups multiple deployments and gives a single pane for larger estates. GoldenGate Veridata, a separate but related product, compares source and target data to validate that replication is actually correct, which is invaluable after a large migration or during periodic audits.

PRACTICAL TAKEAWAY

When planning a new implementation in 2026, default to Microservices Architecture unless there is a hard platform limitation forcing Classic. Oracle's own roadmap, documentation emphasis, and new feature delivery are all centered on Microservices; Classic still works but receives little forward investment.

Chapter 2: Classic Architecture Explained

Classic Architecture is built from a small number of moving parts, and once these are understood clearly, everything else in GoldenGate becomes easier to reason about. The four building blocks are Manager, Extract, Trail Files, and Replicat, with an optional Data Pump sitting between Extract and the network.

2.1 Manager Process

Manager is the supervisor process on every GoldenGate installation, Classic or Microservices-adjacent utility hosts included. It starts and restarts Extract and Replicat processes according to AUTOSTART and AUTORESTART parameters, purges old trail files based on retention rules, manages port allocation for remote connections, and generates event and error records that show up in the GGSCI error log. A common production mistake is treating Manager as unimportant because it does not move data itself — in reality, if Manager is down, a crashed Extract will not restart automatically, and trail files can silently fill up disk because purge rules never execute.

```
-- Typical Manager parameter file (mgr.prm)
PORT 7809
AUTOSTART EXTRACT *
AUTORESTART EXTRACT *, RETRIES 5, WAITMINUTES 2, RESETMINUTES 60
PURGEOLDEXTRACTS ./dirdat/*, USECHECKPOINTS, MINKEEPHOURS 24
LAGREPORTHOURS 1
LAGINFOMINUTES 30
LAGCRITICALMINUTES 60
```

2.2 Extract Process

Extract reads changes from the database and writes them into trail files. In Classic Capture mode it reads the redo log directly from disk, parses the internal redo records, and reconstructs logical change records — inserts, updates, deletes, and DDL where supported. It maintains a checkpoint that records exactly how far it has read, so that after a restart it resumes from the correct SCN instead of re-reading data or skipping changes.

A subtle but important point that new administrators often miss: Extract does not read from the buffer cache and does not touch application tables directly, apart from an initial read during instantiation. Everything after that comes from the redo stream. This is precisely why GoldenGate has such a low footprint on the source database compared to trigger based replication.

2.3 Data Pump

A Data Pump is simply a second Extract, typically configured with PASSTHRU, that reads the local trail written by the primary Extract and forwards it across the network to the remote trail on the target. The reason to use a pump instead of having the primary Extract write directly across the network is resilience: if the network link goes down, the primary Extract keeps capturing into the local trail with no data loss, and the pump simply catches up once connectivity returns. Without a pump, a prolonged network outage can force the primary Extract to buffer excessively or, in the worst case, fail.

> **PRODUCTION WARNING**
> Skipping the Data Pump entirely to save one process is a common shortcut on smaller systems, but it removes the buffer against network interruptions. For any WAN link between source and target data centers, a pump is considered mandatory practice, not optional.

2.4 Trail Files and Trail Headers

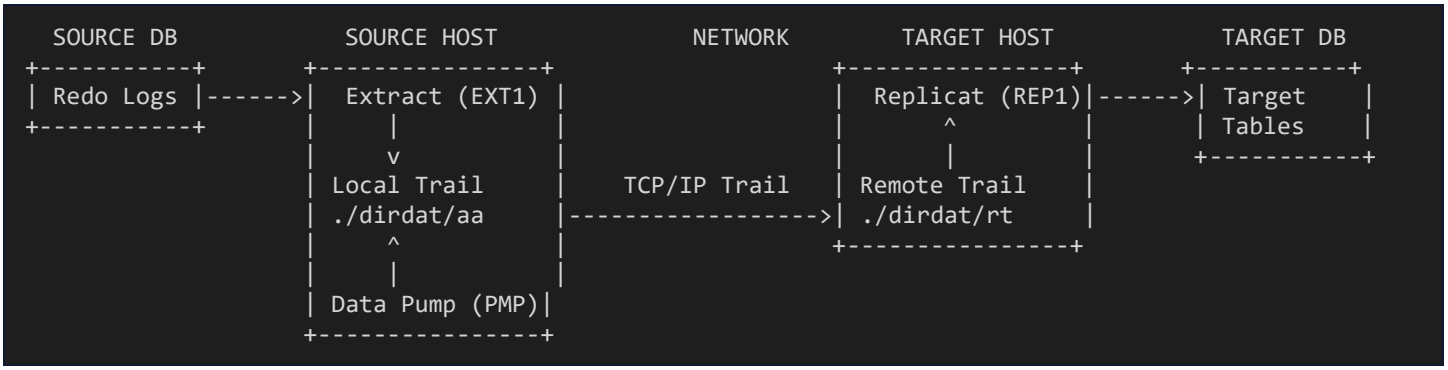
Trail files are the sequential binary files that hold captured change records. Each trail file begins with a trail header that records information such as the GoldenGate version that wrote it, character set, and compatibility flags — this is why moving trail files between mismatched GoldenGate versions can fail outright rather than just producing wrong data. Trail files are named with a two-character prefix followed by a six digit sequence number, for example aa000001, aa000002, and so on, and Extract rolls over to a new file once MEGABYTES or similar sizing parameters are hit.

```
EXTRACT EXT1
USERIDALIAS ggadmin
EXTTRAIL ./dirdat/aa
TABLE HR.EMPLOYEES;
TABLE HR.DEPARTMENTS;
```

2.5 Replicat Process (Classic Overview)

Replicat reads records from the trail file on the target side and applies them using standard SQL against the target database. In Classic Replicat this happens transaction by transaction, honoring commit boundaries exactly as they occurred on the source, which is what guarantees transactional consistency on the target. Detailed internals of each Replicat type are covered in Chapter 6.

2.6 End-to-End Flow



This picture holds true whether the deployment is Classic or Microservices; what changes between the two is how these processes are managed and monitored, not the fundamental data flow.

Chapter 3: Microservices Architecture in Detail

Microservices Architecture reorganizes the same conceptual pipeline — Extract, trail, Replicat — under a set of REST-enabled services that can be managed, secured, and monitored consistently across an entire estate. Understanding each service individually removes a lot of the confusion administrators face when moving from Classic.

3.1 Service Manager

Service Manager is the single process that starts on a host and is responsible for launching, stopping, and monitoring one or more deployments. It listens on a configured port, commonly 9100 or an administrator-chosen value, and exposes both a browser-based console and REST endpoints. Where Manager in Classic supervises processes for one instance, Service Manager can host multiple isolated deployments on the same server, each with its own users, ports, and configuration, which is extremely useful for consolidating many small replication workloads onto shared infrastructure.

3.2 Deployments

A deployment is an isolated GoldenGate environment under a Service Manager, with its own directory structure, its own set of Extract and Replicat processes, and its own security realm. Multiple deployments on one host let different application teams or different projects share hardware without their configurations, trail files, or credentials colliding. Each deployment maps to a dedicated OS user or a distinct working directory depending on how it was provisioned.

3.3 Administration Server

The Administration Server is the service Admin Client actually talks to. It exposes REST APIs for creating and managing Extract, Replicat, parameter files, and credentials, and it is the service responsible for validating configuration before processes start. Nearly every administrative action in Microservices Architecture — adding a table mapping, checking process status, starting or stopping a process — ultimately becomes a REST call against the Administration Server, whether issued through Admin Client, the browser console, or a custom automation script.

```
curl -X GET https://ggnode1:9010/services/v2/extracts/EXT1 \
     -u oggadmin:******* -k

Sample (trimmed) JSON response:
{
  "name": "EXT1",
  "processType": "EXTRACT",
  "status": "RUNNING",
  "checkpoint": { "scn": "18234567890", "timestamp": "2026-06-30T04:12:11Z" }
}
```

3.4 Distribution Server and Receiver Server

The Distribution Server takes over the job that a Data Pump performed in Classic Architecture — it reads the local trail and pushes records to the target across the network, but now does so as a manageable service with its own REST endpoints and TLS configuration rather than a bare Extract process. The Receiver Server on the target side accepts the incoming stream and writes it into the remote trail, replacing the raw collector process from Classic days. Together, Distribution Server and Receiver Server give administrators visibility into network throughput, retry behaviour, and TLS handshake status that was much harder to observe in Classic.

3.5 Performance Metrics Server

Performance Metrics Server collects statistics from every running process in a deployment — lag, throughput, operations per second, and latency — and exposes them through REST so external monitoring tools such as Prometheus-based dashboards, OCI Monitoring, or GoldenGate Monitor itself can pull the numbers without having to log in to each host and run GGSCI or Admin Client commands manually.

Microservice	Classic Equivalent	Primary Responsibility
Service Manager	N/A (new concept)	Starts, stops, and supervises deployments and their services
Administration Server	GGSCI + Manager combined	Configuration, process control, credential management
Distribution Server	Data Pump	Sends trail data to a remote Receiver Server across the network
Receiver Server	Collector	Accepts incoming trail data and writes the remote trail
Performance Metrics Server	GGSCI STATS / LAG output	Aggregates and exposes performance and lag metrics via REST

3.6 Deployment Directory Structure

Every deployment keeps a directory layout very close to the familiar Classic layout — dirprm for parameter files, dirdat for trail files, dirrpt for report files, dirchk for checkpoint files — which is a deliberate design choice so that experienced administrators are not disoriented when they open a Microservices deployment for the first time.

> **MIGRATION NOTE**
> Moving from Classic to Microservices does not require rewriting parameter files from scratch. Most TABLE, MAP, and transformation syntax carries forward unchanged. What changes is how processes are created, started, and monitored — through Admin Client and REST rather than GGSCI talking directly to OS-level processes.

Chapter 4: Extract Internals — Classic Capture vs. Integrated Capture

Capture is the single most performance-sensitive part of a GoldenGate deployment because it runs continuously against a live production redo stream. Choosing between Classic Capture and Integrated Capture, and understanding how each actually works underneath, has direct consequences for CPU usage on the source, RAC behaviour, and the range of data types supported.

4.1 Classic Capture — How It Actually Reads Redo

Classic Capture opens the online redo log files and archived redo log files directly from the file system, using the same physical files that Oracle itself writes for recovery purposes. Extract parses each redo record, decodes the internal Oracle-specific format, and determines whether the change belongs to a table that is configured for capture. Because this parsing logic lives inside the GoldenGate binary rather than the database kernel, Oracle occasionally changes internal redo formats between patch sets in ways that require a corresponding GoldenGate patch to keep parsing correct — this is one of the operational realities that makes strict compatibility matrix checking important before applying a database patch.

On RAC systems, Classic Capture needs to merge redo coming from multiple instances, each writing its own thread of redo, into a single time-ordered stream. This merge logic works, but it adds complexity and a small amount of additional lag compared to a single-instance database.

4.2 Integrated Capture — Moving the Work Into the Kernel

Integrated Capture, in contrast, runs as a background capture process inside the Oracle database itself, using the same logical change record infrastructure that powers LogMiner. Extract, running outside the database, simply requests change records from this in-database capture process through a well-defined interface rather than parsing raw redo bytes itself. This has several advantages: Oracle can guarantee that redo format changes are handled internally without requiring a GoldenGate patch in lockstep, RAC redo merging is handled natively by the database, and compressed table support, which was historically painful under Classic Capture, works cleanly.

The tradeoff is that Integrated Capture consumes a small amount of additional CPU and SGA/PGA memory inside the database itself, since the capture process is now a database session rather than a purely external OS process. For most modern hardware this overhead is negligible, which is why Oracle recommends Integrated Capture as the default choice for Oracle-to-Oracle replication in current releases.

Aspect	Classic Capture	Integrated Capture
Where it runs	External OS process reading redo files directly	Inside the database using LogMiner-based infrastructure
RAC redo merge	Handled by Extract, added complexity	Handled natively by the database
Compressed tables	Limited or problematic support	Fully supported
Resource footprint	Lower database-side load, higher external parsing effort	Slight increase in database CPU / memory, simpler externally
Oracle's current guidance	Legacy, still supported on many platforms	Recommended default for Oracle sources

4.3 Registering Integrated Capture

Before an Extract can run in Integrated Capture mode, it must be registered with the database using DBMS_CAPTURE_ADM or the equivalent GGSCI/Admin Client REGISTER command, which creates the internal capture process and associates it with the Extract group name.

```
GGSCI> DBLOGIN USERIDALIAS ggadmin
GGSCI> REGISTER EXTRACT EXT1 DATABASE
GGSCI> ADD EXTRACT EXT1, INTEGRATED TRANLOG, BEGIN NOW
GGSCI> ADD EXTTRAIL ./dirdat/aa, EXTRACT EXT1
```

4.4 Checkpoints During Capture

Extract maintains its own checkpoint independent of the database, recording the exact SCN and internal position it has processed. This checkpoint is what allows Extract to survive a restart, a host reboot, or a Manager-triggered AUTORESTART without reprocessing already-captured changes or losing anything in between. Checkpoint internals and the corresponding checkpoint tables used on the Replicat side are discussed fully in Chapter 7.

> **BEST PRACTICE**
> Whenever Oracle Database is patched, cross-check the My Oracle Support certification matrix for GoldenGate against the new database patch level before assuming Extract will keep running without issue, particularly for Classic Capture installations. Integrated Capture reduces but does not completely eliminate this risk.

4.5 Initial Load Considerations

Extract by itself only captures ongoing changes; it does not move the existing data that was already in the tables before capture started. An initial load — using Oracle Data Pump export/import, direct bulk copy through an Initial Load Extract and Replicat pair, or a database link based approach — is required to seed the target with a consistent starting point. The BEGIN NOW or BEGIN <timestamp> position on Extract must line up carefully with the SCN at which the initial load was taken, otherwise rows can be duplicated or silently missed.

Chapter 5: Trail Files, Compression, Encryption, and TLS

5.1 Trail File Internal Structure

A trail file is a sequential binary file made up of a trail header followed by a stream of records, where each record represents one logical change — an insert, update, delete, or a piece of DDL metadata. Every record carries a header with the transaction identifier, commit SCN, timestamp, and table identifier, followed by the before and after column images depending on what was configured for capture. Because trail files are self-describing through their header and record structure, tools such as Logdump can open and interpret them without needing a live database connection, which is invaluable during troubleshooting.

5.2 Trail File Naming and Rollover

Trail files use a short prefix, typically two characters chosen by the administrator such as aa or rt, followed by a six digit sequence number that increments as files roll over. Rollover happens automatically once a file reaches the configured size limit.

```
EXTRACT EXT1
EXTTRAIL ./dirdat/aa, MEGABYTES 200

Resulting files as capture proceeds:
./dirdat/aa000000
./dirdat/aa000001
./dirdat/aa000002
```

5.3 Trail Compression

Trail compression reduces the amount of data that must be transmitted across the network and stored on disk, which becomes significant on high-volume systems where trail files can otherwise consume large amounts of storage very quickly. Compression is commonly applied on the Data Pump or Distribution Server side, trading a small amount of CPU for a meaningful reduction in network bandwidth, which is usually a good trade on WAN links between data centers.

5.4 Encryption of Trail Data

GoldenGate supports encrypting trail files at rest and encrypting data in transit between Extract/Pump and the target Collector or Receiver Server. At-rest encryption protects trail files sitting on disk from being read by anyone with file system access but without the GoldenGate encryption key, which matters because trail files contain full row images of production data, including sensitive columns, unless explicit column exclusion or masking is configured.

5.5 TLS for Network Transport

For data in transit, GoldenGate supports TLS between the source-side sending process and the target-side receiving process. In Classic Architecture this is configured through the ENCRYPT parameter along with the relevant TLS options on Manager. In Microservices Architecture, TLS is configured at the Distribution Server and Receiver Server level, and typically also protects the REST API endpoints themselves, since Admin Client and any monitoring tool now talk to these services over HTTPS rather than a raw TCP port.

> **SECURITY WARNING**
> Sending trail data across a public or semi-trusted network without TLS is a serious exposure, since trail records contain full column values, including anything that has not been explicitly masked. Any GoldenGate link that crosses outside a trusted, physically controlled network segment should have TLS enabled, no exceptions.

5.6 Credential Store

Older GoldenGate configurations embedded database usernames and passwords directly inside parameter files in clear text, which was a well-known security weakness. Credential Store solves this by storing an alias mapped to an encrypted username and password combination, so parameter files reference USERIDALIAS rather than an actual password.

```
GGSCI> ADD CREDENTIALSTORE
GGSCI> ALTER CREDENTIALSTORE ADD USER ggadmin@orclpdb1, ALIAS ggadmin
-- Parameter file then simply references:
USERIDALIAS ggadmin
```

5.7 Discard File Relationship to Trail Data

When Replicat cannot apply a record — perhaps due to a constraint violation or a data type mismatch — the offending record is written to a discard file rather than silently dropped or allowed to stop the entire process indefinitely, depending on how HANDLECOLLISIONS and related error handling parameters are configured. Discard files are examined in detail in Chapter 14 on troubleshooting, but it is worth noting here that they are, at their core, simply readable extracts of the same trail record structure described in this chapter.

Chapter 6: Replicat Architecture — Classic, Coordinated, Integrated, and Parallel

Replicat is where transactions captured on the source finally get applied on the target, and this is usually where production issues surface first, because apply throughput on the target is very often the bottleneck in an end-to-end replication pipeline rather than capture on the source.

6.1 Classic Replicat

Classic Replicat reads trail records in the exact order they were committed on the source and applies them serially against the target using standard SQL — INSERT, UPDATE, DELETE statements generated from the trail records. Because it processes one transaction at a time, in commit order, it is the easiest Replicat type to reason about and troubleshoot, and it is the safest default when transactional dependency across tables must be preserved with zero ambiguity. Its obvious limitation is throughput: a single thread applying serially cannot use more than one CPU core's worth of apply capacity, which becomes a real ceiling on busy OLTP targets.

```
REPLICAT REP1
USERIDALIAS ggadmin
ASSUMETARGETDEFS
MAP HR.EMPLOYEES, TARGET HR.EMPLOYEES;
MAP HR.DEPARTMENTS, TARGET HR.DEPARTMENTS;
```

6.2 Coordinated Replicat

Coordinated Replicat introduces manual parallelism. The administrator defines a number of threads, and a coordinator thread hands transactions off to worker threads based on a hash computed from key columns, ensuring that changes affecting the same row consistently land on the same thread and therefore stay in correct order relative to each other. Cross-thread dependencies, such as a foreign key relationship spanning two tables assigned to different threads, must be handled carefully through the same MAP definitions and thread assignment rules, because getting this wrong can apply a child row before its parent exists, causing constraint failures.

```
REPLICAT REP1
USERIDALIAS ggadmin
COORDINATED MAXTHREADS 4
MAP HR.EMPLOYEES, TARGET HR.EMPLOYEES, THREAD(1,4, EMPLOYEE_ID);
MAP HR.DEPARTMENTS, TARGET HR.DEPARTMENTS, THREAD(1,4, DEPARTMENT_ID);
```

6.3 Integrated Replicat

Integrated Replicat hands the incoming logical change records to an inbound server running inside the target Oracle database. The database itself then decides how to parallelize the apply work, using its own internal knowledge of table dependencies, constraints, and transaction boundaries to safely run independent transactions concurrently while keeping dependent ones properly ordered. This removes the manual thread-and-hash-key design burden that Coordinated Replicat placed on the administrator, and it is the standard recommendation for Oracle-to-Oracle replication on current releases.

```
GGSCI> DBLOGIN USERIDALIAS ggadmin
GGSCI> ADD REPLICAT REP1, INTEGRATED, EXTTRAIL ./dirdat/rt
-- Parallelism is then tuned through the inbound server, not manual THREAD() clauses
GGSCI> ALTER REPLICAT REP1, PARALLELISM 4
```

6.4 Parallel Replicat

Parallel Replicat goes a step further and can be used for both Oracle and select non-Oracle targets. Instead of relying on the target database's own inbound server, Parallel Replicat itself builds a dependency graph across incoming transactions, groups independent ones together, and applies them concurrently across multiple apply threads that it manages internally. It removes the need for the administrator to size Coordinated Replicat threads manually, while remaining usable on targets that do not have an Oracle-specific inbound server available.

Feature	Coordinated Replicat	Parallel Replicat
Thread sizing	Manual, chosen by administrator	Automatic, sized dynamically
Row-to-thread assignment	Hash on key column, defined by DBA	Dependency graph built automatically
Target database support	Broad, works on most JDBC-capable targets	Oracle and a growing list of non-Oracle targets
Administration effort	Higher — requires careful THREAD() design	Lower — largely self-tuning

6.5 Transaction Grouping in Apply

Regardless of Replicat type, GROUPTRANSOPS controls how many source transactions are grouped together into a single target transaction/commit, which reduces the number of commit round trips against the target and materially improves throughput on high-latency or high-volume links. BATCHSQL groups similar SQL operations together to reduce the number of individual execute calls, which again reduces overhead, particularly beneficial for bulk load style workloads with large numbers of inserts.

> **TUNING TIP**
> Raising GROUPTRANSOPS and enabling BATCHSQL together is one of the single most effective, low-risk performance changes available on a lagging Replicat, provided the target can tolerate slightly larger transaction sizes. Always validate against a realistic load test before pushing the change into production.

6.6 How Rollback Behaves in GoldenGate

A source-side rollback never becomes a trail record in the first place, because GoldenGate only captures committed data — uncommitted changes that are later rolled back are simply never written to the trail. This is a frequent point of confusion for administrators new to log-based replication who expect to see some sort of compensating record; there is none, because there was never anything to compensate for on the target.

Chapter 7: Checkpoints and Recovery Internals

Checkpointing is what makes GoldenGate resilient to restarts, crashes, and planned maintenance without either losing data or duplicating it. Both Extract and Replicat maintain checkpoints, but they track fundamentally different kinds of position information.

7.1 Extract Checkpoint Files

Extract stores its checkpoint in a binary checkpoint file located under dirchk, recording the exact SCN and internal redo position it has processed, along with the position within the trail file it last wrote to. On restart, Extract reads this checkpoint file and resumes reading redo from precisely that point, guaranteeing exactly-once capture of every committed change, no more and no less.

7.2 Replicat Checkpoint Files and Checkpoint Tables

Replicat similarly keeps a checkpoint file recording its position within the trail file it is reading. However, Replicat also strongly benefits from a checkpoint table inside the target database itself. The checkpoint table stores the same positional information but does so transactionally, as part of the same commit that applies the actual data changes. This matters because if Replicat crashes between updating its local checkpoint file and committing to the target, a file-only checkpoint could become slightly inconsistent with what was actually committed. A checkpoint table, updated in the same transaction as the data, eliminates this narrow window entirely and is considered mandatory in any serious production deployment.

```
GGSCI> DBLOGIN USERIDALIAS ggadmin
GGSCI> ADD CHECKPOINTTABLE HR.GGS_CHECKPOINT
GGSCI> ADD REPLICAT REP1, EXTTRAIL ./dirdat/rt, CHECKPOINTTABLE HR.GGS_CHECKPOINT
```

PRODUCTION WARNING

Running Replicat without a checkpoint table, relying only on the local checkpoint file, is a shortcut sometimes seen in quickly built test environments. Carrying that same shortcut into production increases the risk of subtle checkpoint drift after an unclean shutdown, and it should be corrected before go-live.

7.3 Checkpoint Recovery Flow

When a Replicat restarts after a crash, it first reads the checkpoint table (if configured) to determine the exact trail position of the last fully committed target transaction, then positions itself at that point in the trail and resumes applying from there. Because the checkpoint update and the data commit happen in the same target transaction, the recovered position is always exactly correct — there is no possibility of the checkpoint claiming a transaction was applied when it actually was not, or vice versa.

```
Trail Records:  ... T100(commit) | T101(commit) | T102(commit) | T103(in-progress)
                                                        ^
                                        Checkpoint table records: last applied = T102

On restart, Replicat resumes reading from immediately after T102,
reapplying T103 onward with no duplication of T100-T102.
```

7.4 Checkpoint Recovery for Extract

Extract's recovery is similar in spirit but does not need a database checkpoint table on the source, because Extract's own checkpoint file, combined with the redo log's inherent durability, is enough to guarantee no missed or duplicated capture. What Extract's checkpoint does need to survive intact is the checkpoint file itself; losing this file without a valid backup effectively forces a fresh BEGIN position, which usually means re-instantiating the target from scratch to avoid gaps.

7.5 Backing Up Checkpoint Information

Because checkpoint files are small but critical, they should be included in regular backup routines for the GoldenGate home, alongside parameter files and the credential store. Losing dirchk without a backup, especially for Extract, is one of the more painful recovery scenarios an administrator can face, since restoring from database backups alone is not sufficient to reconstruct exact capture position.

7.6 Interview Angle

A frequently asked interview question is why GoldenGate needs both a checkpoint file and a checkpoint table for Replicat rather than just one. The correct answer centers on transactional consistency: the checkpoint file alone cannot guarantee atomicity with the actual data commit, while the checkpoint table, committed together with the data, closes that gap completely.

Chapter 8: DDL Replication

Replicating structural changes — new columns, altered data types, renamed tables — is fundamentally different from replicating row data, and GoldenGate handles this through a dedicated DDL replication mechanism rather than simply capturing DDL as if it were another type of DML.

8.1 How DDL Capture Works Internally

On the source, when DDL replication is enabled, Oracle GoldenGate relies on database-side triggers and internal support objects (installed as part of DDL support setup) that detect a DDL event, record its text along with relevant metadata such as the object name, schema, and timestamp, into a dedicated history table. Extract then reads this DDL metadata, translates it into a trail record of DDL type, and this flows through the pipeline just like a row change, arriving at Replicat in correct chronological order relative to any DML that came before or after it.

8.2 Why Metadata Matters

A plain text DDL statement alone is not always enough for Replicat to apply correctly on the target, particularly when source and target schemas are not identical (different tablespace names, different column orders, or additional target-only columns). This is why the DDL trail record carries structured metadata describing exactly what changed, not just the raw SQL text, allowing Replicat to translate the change appropriately for the target schema shape rather than blindly re-running the source's exact SQL statement.

8.3 DDL INCLUDE and DDL EXCLUDE

DDL replication for every single object in a database is rarely desirable, so DDL INCLUDE and DDL EXCLUDE parameters let the administrator scope exactly which schemas, objects, or operation types get replicated. Exclusions are commonly applied to housekeeping objects, temporary tables, or schemas outside the replication scope entirely, since capturing DDL for objects never touched by DML replication just adds unnecessary overhead and risk.

```
EXTRACT EXT1
DDL INCLUDE MAPPED
DDL EXCLUDE INSTR 'GG_TEMP', EXCLUDE OBJTYPE 'INDEX'
TABLE HR.*;
```

8.4 Common DDL Replication Limitations

- Certain complex DDL operations, particularly ones involving partition maintenance or specific object types, may need to be manually excluded and applied out of band during a coordinated maintenance window instead of relying on automatic replication.
- DDL replication across heterogeneous databases (for example Oracle to PostgreSQL) is generally far more limited than Oracle-to-Oracle, and structural changes usually need to be applied manually on the target in mixed-platform topologies.
- A very large batch of DDL statements executed in a short window (such as a major application release) can momentarily increase Extract and Replicat lag while the metadata is processed, and this should be planned for rather than treated as an unexpected incident.

8.5 Old Behaviour vs New Behaviour

In early GoldenGate releases, DDL replication was more fragile and required a fairly involved manual setup of the DDL support schema, triggers, and history objects, and it worked reliably only for a fairly narrow set of DDL operations. In current 19c and later releases, particularly under Integrated Capture, DDL handling is considerably more robust and the setup is largely automated through the standard AddTrandata and DDL configuration scripts shipped with the GoldenGate installation, reflecting years of Oracle hardening this specific feature based on production feedback.

> **BEST PRACTICE**
> Always test DDL replication thoroughly in a staging environment that mirrors production schema complexity before relying on it during a live application release. Application releases are exactly the moment DDL replication is needed most and exactly the moment least tolerant of surprises.

Chapter 9: Parameter Files and Data Selection — TABLE, MAP, and Beyond

The bulk of day-to-day GoldenGate administration happens inside parameter files, and a strong working knowledge of TABLE, MAP, and the surrounding transformation and filtering parameters is what separates someone who can keep an existing configuration running from someone who can design a new one confidently.

9.1 TABLE and MAP — The Core Difference

TABLE is used on the Extract side to specify which source objects should be captured. MAP is used on the Replicat side to specify how a captured source object corresponds to a target object, including any column-level transformation. A very common beginner mistake is trying to use MAP inside an Extract parameter file, or TABLE inside a Replicat parameter file — the parameters are not interchangeable and each only belongs on its respective side of the pipeline.

```
-- Extract side
TABLE HR.EMPLOYEES;
TABLE HR.DEPARTMENTS, FILTER (DEPARTMENT_ID > 100);

-- Replicat side
MAP HR.EMPLOYEES, TARGET HR_STAGE.EMPLOYEES, COLMAP (USEDEFAULTS, LOAD_DATE = @DATENOW());
MAP HR.DEPARTMENTS, TARGET HR_STAGE.DEPARTMENTS;
```

9.2 COLMAP for Column Level Transformation

COLMAP defines exactly how source columns map to target columns, which becomes necessary whenever column names differ, column order differs, or a computed or derived value needs to be populated on the target that does not exist on the source at all. USEDEFAULTS tells Replicat to map identically named columns automatically, so COLMAP is typically only needed to describe the exceptions rather than every single column explicitly.

9.3 FILTER and WHERE

FILTER, generally used on the Extract side, restricts which rows get captured in the first place based on a row-level condition, meaning excluded rows never even make it into the trail file. WHERE, usable on the Replicat side within a MAP statement, restricts which rows get applied on the target even though they may have already been captured into the trail. The distinction matters for performance and for auditing purposes: FILTER reduces trail volume and network traffic, while WHERE only reduces what gets applied but the excluded rows still travel through the pipeline first.

9.4 PASSTHRU and NOPASSTHRU

PASSTHRU tells a Data Pump (or Distribution Server) to forward trail records without opening a database connection to re-check the metadata, which is faster and is the standard choice for pumps that are not doing any transformation. NOPASSTHRU, the default in some configurations, causes the pump to validate table definitions against a database connection, which is only necessary when the pump itself is performing some transformation work rather than simply relaying records untouched.

9.5 Performance and Behaviour Parameters

Parameter	Purpose
BATCHSQL	Groups similar SQL operations into batches to reduce execute round trips on apply.
GROUPTRANSOPS	Groups multiple source transactions into a single target commit to reduce commit overhead.
CACHEMGR	Controls the memory cache GoldenGate uses to buffer transaction data, tunable for very large or long-running transactions.
FETCHOPTIONS	Controls whether and how Extract fetches missing column values (for example when using compressed tables) directly from the database rather than relying solely on redo.
EOFDELAY	Sets the delay Extract or Replicat waits before checking for new data once it reaches the end of a file, balancing responsiveness against unnecessary CPU polling.
HANDLECOLLISIONS	Tells Replicat to tolerate certain conflicts, such as inserting a row that already exists, typically used temporarily during an initial load reconciliation window.

9.6 SQLEXEC

SQLEXEC lets Replicat (or occasionally Extract) call a stored procedure or run an inline SQL statement as part of processing a record, which is useful for scenarios such as invoking existing business logic that must run whenever certain data changes, or performing a lookup that a straightforward COLMAP cannot express. SQLEXEC does carry a real performance cost, since it introduces an additional database round trip per invocation, so it should be reserved for genuine business requirements rather than used as a convenience shortcut for something COLMAP could otherwise handle natively.

```
MAP HR.EMPLOYEES, TARGET HR.EMPLOYEES,
SQLEXEC (SPNAME audit_pkg.log_employee_change,
PARAMS (p_emp_id = EMPLOYEE_ID, p_action = '@GETENV ("GGHEADER", "OPTYPE")'));
```

9.7 Macros and User Exits

Macros allow a block of repeated parameter syntax to be defined once and reused across many TABLE or MAP statements, which keeps large parameter files maintainable when dozens or hundreds of similarly structured mappings are needed. User Exits go a step further, allowing custom C-language routines to hook directly into GoldenGate's processing at defined points, used for advanced scenarios such as custom data transformation logic that cannot be expressed through standard parameters at all.

9.8 Tokens

Tokens allow additional metadata to be captured on the Extract side and carried through the trail record for later use on the Replicat side, commonly used to pass along audit information such as the originating application user or transaction identifier that would not otherwise be visible on the target.

Chapter 10: Security — Credential Store, Encryption, and TLS in Practice

Security in a replication pipeline deserves the same rigor as security anywhere else in the data estate, because a compromised GoldenGate configuration effectively exposes a live, continuously updated copy of production data.

10.1 Credential Store Deep Dive

Credential Store, introduced to remove clear text passwords from parameter files, stores encrypted username and password pairs under an alias inside a wallet-like structure on disk. Every USERIDALIAS reference in a parameter file resolves against this store at runtime, meaning the actual password never appears anywhere in a parameter file, GGSCI history, or process listing.

```
GGSCI> ADD CREDENTIALSTORE
GGSCI> ALTER CREDENTIALSTORE ADD USER ggadmin@orclpdb1, PASSWORD ******, ALIAS gg_src
GGSCI> ALTER CREDENTIALSTORE ADD USER ggadmin@orclpdb2, PASSWORD ******, ALIAS gg_tgt
GGSCI> INFO CREDENTIALSTORE
```

10.2 Encryption Key Management

Encrypting trail files at rest requires generating and securely storing an encryption key or master key, depending on the release and whether Oracle Wallet integration is used. Losing this key permanently makes any encrypted trail file unreadable, which means key backup procedures deserve the same seriousness as database backup procedures — an encrypted trail file with a lost key is, for practical purposes, a corrupted trail file.

10.3 TLS Configuration in Classic vs Microservices

In Classic Architecture, TLS between the pump and the remote collector is configured through the ENCRYPT parameter on the RMTHOST or TCP connection definition, referencing a certificate and key pair configured for Manager. In Microservices Architecture, TLS is configured more holistically: the Service Manager listener itself, the Administration Server REST endpoint, and the Distribution Server to Receiver Server data channel can each be independently secured with certificates, giving finer control but also more moving parts to configure correctly.

PRODUCTION WARNING

A very common misconfiguration is enabling TLS on the REST/administration layer while leaving the actual Distribution Server to Receiver Server data channel unencrypted, under the mistaken assumption that securing the management interface automatically secures the data path as well. Both layers must be configured independently.

10.4 Role Separation and GGSCI/Admin Client Privileges

Database privileges granted to the GoldenGate administration user should follow least-privilege principles: capture generally needs SELECT and specific GoldenGate-related roles rather than broad DBA privileges, and apply needs INSERT/UPDATE/DELETE on the specific mapped objects rather than schema-wide grants. In Microservices deployments, an additional layer of role separation exists at the Administration Server level itself, where different named users can be restricted to read-only monitoring versus full administrative control over process configuration.

10.5 Auditing Considerations

Because trail files and, by extension, discard files, can contain full row images including sensitive columns, any data masking or column exclusion requirements dictated by compliance policy must be applied at the TABLE or MAP level, using COLMAP or explicit column exclusion, rather than assumed to be handled elsewhere in the pipeline by default. GoldenGate does not automatically redact sensitive data; this is entirely the administrator's responsibility to configure explicitly.

Chapter 11: Monitoring, Lag Management, and the REST API

A GoldenGate deployment that is not actively monitored will eventually cause a production incident, usually in the form of unnoticed lag that grows silently until it becomes a business-visible problem, such as a reporting system showing stale data or a downstream application acting on outdated records.

11.1 Heartbeat Tables

Heartbeat monitoring works by periodically inserting a timestamped row on the source, letting it flow through the entire replication pipeline like any other change, and then comparing the timestamp against the time it was actually applied on the target. The difference between these two timestamps is true end-to-end lag, which is a far more meaningful number than simply checking whether Extract or Replicat individually report themselves as current, since a healthy-looking Extract with a badly lagging Replicat downstream would otherwise go unnoticed.

```
GGSCI> ADD HEARTBEATTABLE HR.GG_HEARTBEAT
GGSCI> INFO HEARTBEATTABLE

-- Typical monitoring query against the heartbeat history table
SELECT source_commit_timestamp, target_commit_timestamp,
       (target_commit_timestamp - source_commit_timestamp) AS lag_interval
FROM   GG_HEARTBEAT_HISTORY
ORDER  BY target_commit_timestamp DESC
FETCH FIRST 10 ROWS ONLY;
```

11.2 LAG and STATS Commands

GGSCI (and Admin Client) provide direct commands to check lag and throughput without waiting for a heartbeat cycle. LAG EXTRACT and LAG REPLICAT report the difference between the current time and the timestamp of the last processed record, while STATS reports operation counts, useful for confirming that a process is actually doing meaningful work rather than merely appearing to run.

```
GGSCI> LAG EXTRACT EXT1
GGSCI> LAG REPLICAT REP1
GGSCI> STATS REPLICAT REP1, TOTALSONLY *
GGSCI> SEND EXTRACT EXT1, STATUS
```

11.3 Auto Restart and Threshold Alerts

AUTORESTART on Manager, seen earlier in Chapter 2, handles transient failures automatically, but it is not a substitute for alerting — repeated automatic restarts of the same process within a short window usually indicate an underlying problem, such as a network flap or a target database issue, that needs human attention rather than being silently absorbed forever. LAGCRITICALMINUTES and LAGINFOMINUTES on Manager define thresholds at which GoldenGate itself raises warning and critical events into the error log, which should be wired into whatever enterprise monitoring or alerting platform is already in use.

11.4 REST API for Monitoring

In Microservices Architecture, the Performance Metrics Server exposes lag, throughput, and status information through REST endpoints, letting external tools poll GoldenGate health without needing to script GGSCI or Admin Client sessions. This is the foundation that products like GoldenGate Monitor and OCI Monitoring integration build upon.

```
curl -X GET https://ggnode1:9011/services/v2/metrics/EXT1/lag \
     -u oggadmin:******* -k

Sample (trimmed) response:
{ "processName": "EXT1", "lagInSeconds": 4, "status": "RUNNING" }
```

11.5 GoldenGate Monitor

GoldenGate Monitor is Oracle's dedicated monitoring product for larger estates, providing dashboards across many deployments, historical lag trending, and alert configuration in a single console, rather than the administrator having to script their own aggregation across dozens of REST endpoints. For smaller estates with only a handful of deployments, direct REST polling into an existing enterprise monitoring tool is often sufficient and avoids introducing another product to license and maintain.

> **BEST PRACTICE**
> Configure heartbeat monitoring on every production pipeline from day one, not as an afterthought added after the first lag-related incident. End-to-end lag visibility is consistently the single most useful piece of health information an administrator can have.

Chapter 12: Conflict Detection, Conflict Resolution, and Loop Detection

Bidirectional and multi-master replication topologies introduce a class of problems that simple one-directional pipelines never encounter: the same row can be changed independently on two sides at nearly the same time, and changes can loop back to their origin if not handled carefully.

12.1 Why Conflicts Happen

In an active-active configuration, both sides accept application writes independently, and network or processing delay means it is entirely possible for the same row to be updated on Site A and Site B within the same short window, each unaware of the other's change. When each side's Replicat eventually receives the other side's update, GoldenGate has to decide what happens: apply it and overwrite the local value, discard it, or apply some merge logic — this decision is exactly what Conflict Detection and Resolution, commonly referred to as CDR, is built to automate.

12.2 Conflict Detection Mechanics

Detection works by comparing the before-image carried in the incoming trail record against the row's current value in the target database at the moment of apply. If they match, there is no conflict — the row has not changed locally since the source captured its version. If they differ, a conflict has genuinely occurred, and the configured resolution policy takes over rather than the operation simply failing outright.

12.3 Common Resolution Policies

Policy	Behaviour
Overwrite / Latest Timestamp Wins	The change with the later timestamp is kept, the earlier one is discarded.
Site Priority	A designated site's changes always win regardless of timestamp, useful when one location is authoritative for a given data domain.
Additive / Delta-based	For numeric columns, deltas are combined rather than one full value overwriting another, appropriate for counters or running totals.
Discard and Log	The conflicting change is not applied and is instead logged for manual review, appropriate for high-value or highly sensitive data where automatic resolution is considered too risky.

12.4 Loop Detection

Without loop detection, a bidirectional configuration would send a change from Site A to Site B, and Site B's own Extract, seeing that change committed locally, would capture it again and send it right back to Site A, creating an infinite loop of the same change bouncing back and forth. GoldenGate prevents this by tagging replicated changes with a marker (commonly implemented through a token or trail record metadata identifying the originating source), and Extract is configured to exclude changes carrying that marker from being re-captured, breaking the loop cleanly.

```
EXTRACT EXTB
-- Exclude transactions that originated from GoldenGate replication itself,
-- preventing a change replicated in from Site A from being sent back out.
TABLE HR.EMPLOYEES, FILTER (@GETENV ('GGENVIRONMENT', 'OGGSOURCE') = 'NONE');
```

12.5 Design Guidance for Active-Active Topologies

- Where possible, design application logic to avoid true multi-master writes to the same row from multiple sites, since even the best automatic conflict resolution is a compromise compared to genuinely avoiding the conflict in the first place.
- Use Site Priority resolution for reference or configuration data that logically belongs to one authoritative location.
- Log every resolved conflict, even when resolution is fully automatic, so that data quality issues arising from unexpected conflict patterns can be investigated after the fact.
- Test loop detection explicitly during setup by making a change on each side and confirming it does not reappear as a duplicate change back at its origin.

Chapter 13: Performance Tuning

Performance tuning in GoldenGate is mostly about finding where in the pipeline time is actually being spent — source-side capture, network transport, or target-side apply — rather than blindly changing parameters and hoping for improvement.

13.1 Diagnosing Where Lag Is Coming From

The first step in any lag investigation is separating Extract lag from Replicat lag using the LAG command covered in Chapter 11. If Extract lag is high while Replicat lag is low, the bottleneck is on the source side, most likely redo generation rate outpacing capture throughput. If Extract lag is low but Replicat lag is high, the bottleneck is almost always on the target side, and tuning effort should focus on apply parallelism and target database performance rather than anything on the source.

13.2 Source-Side Tuning

- Prefer Integrated Capture over Classic Capture for Oracle sources on current releases, since it generally handles high redo generation rates more efficiently and avoids the RAC redo-merge overhead described in Chapter 4.
- Ensure the Extract process has sufficient CACHEMGR memory allocated so that large or long-running transactions do not spill excessively to disk while waiting for a matching commit record.
- Avoid unnecessary FILTER complexity on very high volume tables; simple filters are cheap, but elaborate multi-condition filters evaluated per row can add measurable CPU overhead at scale.

13.3 Network and Trail Tuning

- Enable trail compression on the pump or Distribution Server for WAN links, trading a modest amount of CPU for reduced bandwidth consumption, particularly valuable on constrained or costed network links.
- Size trail files appropriately — very small trail files cause excessive rollover overhead, while very large trail files can slow down certain recovery and Logdump operations; a few hundred megabytes per file is a reasonable general starting point, tuned based on observed volume.
- Monitor TCP-level retransmission and round-trip time between source and target hosts; a healthy GoldenGate configuration cannot outrun an unhealthy network link no matter how well tuned the parameters are.

13.4 Target-Side Apply Tuning

- Move from Classic Replicat to Integrated or Parallel Replicat wherever the target platform supports it, since serial apply is by far the most common ceiling on throughput in real deployments.
- Increase GROUPTRANSOPS and enable BATCHSQL as covered in Chapter 6, validated against realistic transaction sizes rather than assumed blindly.
- Check for missing or inefficient indexes on target tables that Replicat relies on for locating rows to update or delete; a poorly indexed target can make apply dramatically slower than the equivalent workload on the source, where the application's own well-tuned indexes were already in place.
- Watch for excessive supplemental logging or excessive trigger activity on the target caused by the very columns Replicat is updating, which can create a feedback loop of unnecessary overhead.

13.5 A Structured Tuning Checklist

1. Confirm exactly where lag is occurring using LAG EXTRACT and LAG REPLICAT.
2. Check STATS output to confirm the process is actually processing a meaningful operation volume, not stalled entirely.
3. Review the report file for warnings about long-running transactions, checkpoint issues, or repeated errors.
4. Address source-side capture tuning if Extract lag is the primary contributor.
5. Address network and trail tuning if data is captured quickly but arrives slowly on the target.
6. Address target-side apply tuning, including Replicat type and transaction grouping, if data arrives quickly but applies slowly.
7. Re-measure lag after each individual change rather than changing several parameters simultaneously, so the actual effect of each change is clear.

BEST PRACTICE

Always change one tuning parameter at a time and re-measure. Changing GROUPTRANSOPS, BATCHSQL, and Replicat parallelism simultaneously might fix the lag, but nobody will know which change actually mattered the next time a similar problem occurs.

Chapter 14: Troubleshooting — Discard Files, Logdump, and Common Production Errors

14.1 Discard Files

When Replicat encounters a record it cannot apply — a unique constraint violation, a missing parent row, a data type mismatch — the DISCARDFILE parameter determines where that failed record and its associated error detail are written, allowing the process to either continue on to the next record or stop, depending on configuration. Reading a discard file is usually the very first troubleshooting step whenever Replicat abends or repeatedly reports the same apply error, since it shows the exact row and exact error, not just a summary count.

```
REPLICAT REP1
DISCARDFILE ./dirrpt/REP1.dsc, APPEND, MEGABYTES 50

-- Typical discard file entry
2026-06-30 03:14:02  WARNING OGG-01004  Aborted grouped transaction on 'HR.EMPLOYEES',
database error 1 (ORA-00001: unique constraint (HR.EMP_PK) violated).
```

14.2 Logdump

Logdump is the low-level utility used to open and read trail files directly, byte by byte if necessary, entirely independent of a live database connection. It is invaluable when trying to determine whether a particular change actually made it into the trail at all, or when investigating whether a trail file itself is corrupted versus the problem being purely on the apply side.

```
Logdump> OPEN ./dirdat/rt000012
Logdump> GHDR ON
Logdump> DETAIL ON
Logdump> COUNT
Logdump> NEXT
Logdump> POS 0
Logdump> FILTER TABLE HR.EMPLOYEES
Logdump> NEXT
```

14.3 Common Production Errors and Root Causes

Symptom	Likely Root Cause
Extract abends with archived log not found	Archived redo was purged or moved before Extract finished reading it; check RMAN retention against Extract's oldest required SCN.
Replicat abends on unique constraint violation shortly after go-live	Initial load and Extract BEGIN position were not aligned to the same SCN, causing overlapping data.
Sudden lag spike with no volume increase	Target-side blocking session, long-running batch job, or missing index causing slow apply; check target session activity directly.
Extract running but trail file not growing	Table not actually enabled for supplemental logging, or capture filter unintentionally excluding all changes.
Checkpoint appears to roll back after a crash	Checkpoint table missing or not used, forcing reliance on a slightly stale file-based checkpoint; see Chapter 7.

14.4 Error Handling Parameters

HANDLECOLLISIONS, mentioned in Chapter 9, is specifically intended for temporary use during initial load reconciliation, tolerating duplicate inserts and missing rows for deletes/updates that are expected during the cutover window. Leaving HANDLECOLLISIONS enabled permanently in steady-state production is a well-known anti-pattern, since it can mask genuine data integrity problems that would otherwise surface clearly as apply errors.

PRODUCTION WARNING

HANDLECOLLISIONS left enabled long after an initial load has stabilized is one of the more common configuration mistakes found during health checks of long-running GoldenGate environments. It should always be explicitly turned off once the initial reconciliation window has closed.

14.5 A Practical Troubleshooting Sequence

8. Check GGSCI or Admin Client INFO output for process status — RUNNING, STOPPED, ABENDED.
9. Review the report file (dirrpt) for the most recent error or warning entries.
10. If Replicat abended on a specific record, open the discard file to see the exact row and error.
11. If the question is whether a change reached the trail at all, use Logdump to inspect the trail directly.
12. Check LAG and STATS to distinguish a stopped process from a merely slow one.
13. Cross-check any recent source or target database changes — patching, index changes, application releases — against the timing of the issue.

Chapter 15: High Availability, Disaster Recovery, and Zero Downtime Migration

15.1 GoldenGate Alongside Data Guard

A very common enterprise topology runs GoldenGate on top of a Data Guard protected primary database, using GoldenGate for logical, selective replication — often to a different platform, a reporting system, or a different geographic region — while Data Guard handles physical, full-database disaster recovery of the primary itself. When a Data Guard role transition occurs, planned switchover or unplanned failover, Extract needs to follow the redo thread to the new primary, which requires either an Integrated Capture configuration aware of the Data Guard configuration, or a carefully scripted restart procedure pointing Extract at the new primary's redo stream at the correct SCN.

> **DESIGN NOTE**
> Extract should never be left pointing at a former primary that has become a physical standby after a role transition; this appears to work briefly because the standby still has redo, but capture position quickly becomes meaningless once the standby is no longer receiving new production changes as the primary.

15.2 RAC Considerations

On Oracle RAC sources, Extract must see redo from every active instance, which Integrated Capture handles natively as described in Chapter 4. For Classic Capture on RAC, Extract needs explicit THREADS configuration identifying every redo thread it must merge, and any change in the number of active RAC instances requires a corresponding parameter update, which is an operational detail that is easy to overlook during a RAC node addition or removal.

15.3 High Availability for GoldenGate Itself

Beyond protecting the source and target databases, the GoldenGate processes and their configuration also need protection. Common approaches include clustering the GoldenGate home under an OS-level cluster manager so Manager, Extract, and Replicat can fail over to a surviving node, or in Microservices Architecture, deploying Service Manager on infrastructure with its own high availability characteristics and ensuring the deployment's dirprm, dirchk, and dirdat directories sit on shared or replicated storage accessible from any node that might need to take over.

15.4 Zero Downtime Migration Using GoldenGate

A classic pattern for migrating a database with minimal application downtime is to set up GoldenGate replication from the old platform to the new platform well ahead of the actual cutover, let it run and stabilize while the application continues operating against the old platform, validate data consistency (often using GoldenGate Veridata), and then perform a brief cutover window where application traffic is redirected to the new platform only after confirming the last few transactions have fully replicated through.

14. Stand up the target platform and perform an initial load to seed a consistent starting point.
15. Start Extract and Replicat, allow the pipeline to catch up to near-zero lag, and let it run stably for a validation period.
16. Run data comparison checks between source and target to confirm consistency.
17. During a short, well-communicated maintenance window, stop application writes on the source, allow final lag to drain to zero, and confirm via LAG and checkpoint verification that every committed source transaction has been applied.
18. Redirect the application to the new target platform and decommission or repurpose the old platform once confidence in the new platform is established.

15.5 Cross-Platform Migration

Because GoldenGate captures logical change records rather than physical blocks, it is equally capable of replicating between different operating systems, different storage architectures, and even different Oracle Database editions, which makes it one of the standard tools for platform migrations such as moving from on-premises infrastructure to Oracle Cloud Infrastructure, or between different hardware vendors, without requiring the extended downtime a traditional backup-and-restore based migration would need.

15.6 Disaster Recovery Testing

A disaster recovery plan involving GoldenGate should be tested periodically just like any other DR plan — including a genuine simulated failover of the source database, confirming Extract correctly picks up the new primary, and confirming Replicat on the target continues applying without manual intervention beyond what is documented in the runbook. A DR plan that has never actually been executed in a drill carries meaningfully more risk than one that has been tested and refined at least once.

Chapter 16: GoldenGate for Big Data, Kafka, and Heterogeneous Databases

16.1 GoldenGate for Big Data Handlers

GoldenGate for Big Data reads standard trail files, the same format produced by an Oracle-source Extract, and converts the change records into formats and protocols suited to big data and streaming platforms — Kafka topics, HDFS files, Elasticsearch documents, and similar targets — through a set of pluggable handlers. This means the exact same trail file can simultaneously feed a traditional relational target through ordinary Replicat and a Kafka topic through a Big Data handler, without requiring two separate capture configurations on the source.

16.2 Kafka Integration Patterns

The Kafka Handler converts each change record into a message, commonly serialized as JSON or Avro, published to a topic that is often partitioned to correspond to the source table or a chosen key column. This pattern is widely used to feed event-driven microservices architectures, letting downstream services react to database changes in near real time rather than polling the database directly, and it is equally used to feed real time analytics platforms that consume from Kafka rather than querying an operational database.

```
-- Simplified Big Data Replicat properties for Kafka
gg.handlerlist=kafkahandler
gg.handler.kafkahandler.type=kafka
gg.handler.kafkahandler.KafkaProducerConfigFile=custom_kafka_producer.properties
gg.handler.kafkahandler.TopicMappingTemplate=${schemaName}_${tableName}
gg.handler.kafkahandler.format=json
```

16.3 Heterogeneous Database Replication

GoldenGate's heterogeneous support extends capture and apply capability to non-Oracle databases including SQL Server, PostgreSQL, MySQL, and DB2, and select NoSQL-style targets through dedicated handlers where officially supported. The degree of feature parity is not identical across platforms — DDL replication, for instance, is generally far more mature for Oracle-to-Oracle than for Oracle-to-heterogeneous, as noted in Chapter 8, and administrators should always verify current certification and supported feature lists for the specific source and target combination in question rather than assuming full feature parity.

Target Platform	Typical Use Case	Notable Consideration
MySQL	Migrations, reporting offload, cloud database sync	Row-based binary logging must be enabled on MySQL side for capture from MySQL as a source.
PostgreSQL	Heterogeneous migration, hybrid cloud replication	Logical replication slots typically required when PostgreSQL is the source.
SQL Server	Cross-platform migration and coexistence	Change data capture or transaction log reader configuration required on the SQL Server source.
DB2	Mainframe and midrange integration projects	Often paired with specialized capture agents depending on DB2 platform (LUW versus z/OS).
Kafka / Big Data targets	Streaming analytics, event-driven architecture	Delivered via GoldenGate for Big Data handlers rather than a native Replicat.

16.4 OCI GoldenGate

OCI GoldenGate is Oracle's fully managed offering of the Microservices Architecture, removing patching, sizing, and infrastructure management responsibilities from the customer while preserving the same fundamental configuration model — deployments, Extract, Replicat, trail files — that an on-premises Microservices installation uses. This makes it straightforward to extend an existing on-premises topology into the cloud, or to migrate an entire replication workload to a managed service without redesigning the underlying replication logic.

16.5 GoldenGate Hub and Veridata

GoldenGate Hub provides a consolidated, customer-managed console across many deployments, useful for larger on-premises estates that want centralized visibility without adopting a fully managed cloud model. GoldenGate Veridata addresses a different but closely related need: independently comparing source and target data to confirm replication correctness, which is especially valuable immediately after a migration cutover or as a periodic scheduled audit on an ongoing bidirectional or high-value replication pipeline.

Chapter 17: Best Practices, Common Mistakes, and Interview Preparation

17.1 Production Best Practices Checklist

- Use a Data Pump (Classic) or Distribution Server (Microservices) for every link that crosses a network segment, never send directly from the primary Extract across a WAN.
- Always configure a checkpoint table for Replicat, never rely solely on the local checkpoint file in production.
- Enable TLS end-to-end, covering both the administrative/REST layer and the actual data transport layer.
- Store credentials only through Credential Store, never in clear text inside parameter files.
- Configure heartbeat tables and wire lag thresholds into enterprise monitoring from day one.
- Back up dirchk, dirprm, and the credential store as part of routine backup procedures, not as an afterthought.
- Turn off HANDLECOLLISIONS once the initial load reconciliation window has closed.
- Prefer Integrated Capture and Integrated or Parallel Replicat for Oracle-to-Oracle topologies on current releases.
- Test DDL replication and disaster recovery failover procedures in a realistic staging environment, not only in theory.
- Change one performance parameter at a time and re-measure lag before making further changes.

17.2 Common Mistakes Seen in Production

Mistake	Consequence
No Data Pump on a WAN link	Network interruption forces the primary Extract to buffer or fail rather than continuing to capture safely.
Missing checkpoint table on Replicat	Small risk window of checkpoint drift after an unclean shutdown.
HANDLECOLLISIONS left on permanently	Genuine data integrity problems get silently tolerated instead of surfacing as errors.
Mismatched Extract BEGIN position and initial load SCN	Duplicate rows or missing rows shortly after go-live.
Assuming Admin Client is a full drop-in replacement for GGSCI	Certain legacy Classic-only workflows behave unexpectedly under Microservices.
No DR failover drill ever performed	Untested assumptions about Extract following a new primary after failover turn out to be wrong exactly when it matters most.

17.3 Frequently Asked Interview Questions

Q1: What is the difference between Classic Capture and Integrated Capture?

Classic Capture reads redo directly from the file system and parses it inside the Extract process itself, while Integrated Capture runs inside the Oracle database using LogMiner-based infrastructure, giving native RAC redo merging and better handling of compressed tables, at the cost of a small increase in database-side resource usage.

Q2: Why does GoldenGate use a Data Pump instead of sending directly across the network?

A Data Pump isolates the primary Extract from network interruptions. If the network fails, the primary Extract keeps writing to the local trail with zero data loss, and the pump simply resumes forwarding once connectivity is restored.

Q3: Why does Replicat need both a checkpoint file and a checkpoint table?

The checkpoint file alone cannot guarantee atomicity with the actual data commit on the target. A checkpoint table, updated within the same transaction as the applied data, closes that narrow consistency gap entirely.

Q4: What is the practical difference between FILTER and WHERE?

FILTER, applied on the Extract side, prevents rows from ever entering the trail file, reducing both trail volume and network traffic. WHERE, applied on the Replicat side, still allows the row to travel through the pipeline but excludes it from being applied on the target.

Q5: How does GoldenGate prevent an infinite loop in a bidirectional configuration?

Replicated changes are tagged with metadata identifying their origin, and Extract on the receiving side is configured to exclude changes carrying that marker from being re-captured, preventing the same change from bouncing endlessly between sites.

Q6: What happens to a source-side transaction that gets rolled back?

Nothing — GoldenGate only ever captures committed data from the redo stream, so a rolled-back transaction never produces a trail record in the first place and there is nothing to compensate for on the target.

Q7: When would Parallel Replicat be chosen over Integrated Replicat?

Parallel Replicat is useful when the target is not Oracle, or does not have an inbound server available, since it manages its own dependency-based parallelism internally rather than relying on an Oracle-specific database feature.

17.4 Closing Thought on Long-Term Maintainability

A GoldenGate environment that is easy to operate five years from now is one where parameter files are consistently documented, credentials are centrally managed, monitoring is wired into existing enterprise tooling rather than left as a manual daily check, and every significant configuration decision — Replicat type chosen, DDL scope, conflict resolution policy — has a clear, recorded rationale that a new administrator joining the team can read and understand without having to reverse-engineer intent from parameter files alone.

Chapter 18: Upgrade Strategy, Data Warehouse Replication, and Real Time Analytics

18.1 Planning a GoldenGate Upgrade

Upgrading GoldenGate itself, separate from upgrading the underlying database, needs its own careful planning because trail file format compatibility, parameter deprecations, and process behaviour can all shift between major releases. The safe general sequence is to upgrade Replicat-side software first in a rolling fashion where the topology allows it, since a newer Replicat can generally read trail files produced by an older Extract, whereas the reverse is not always guaranteed.

19. Review the release notes and upgrade guide for the specific source and target release combination, paying particular attention to deprecated parameters that older parameter files may still reference.
20. Take a full backup of the GoldenGate home, including dirprm, dirchk, and the credential store, before touching anything.
21. Stop Replicat cleanly, allowing it to reach a clean checkpoint boundary rather than killing the process abruptly.
22. Upgrade the Replicat-side GoldenGate binaries and validate against a copy of the production parameter files in a staging environment first.
23. Repeat the same careful sequence for Extract once Replicat has been confirmed stable on the new release.
24. Monitor lag and error logs closely for the first several hours after each side is upgraded, since subtle behavioural differences often surface only under real production load.

PRODUCTION WARNING

Upgrading Extract and Replicat simultaneously across a large estate in a single change window removes the ability to isolate which side introduced a problem if something goes wrong. A staged, sequential upgrade approach is strongly preferred for anything beyond a small, low-risk environment.

18.2 Migrating from Classic to Microservices Architecture

Moving an existing Classic deployment to Microservices Architecture is best treated as its own project rather than bundled into a routine version upgrade. The general approach is to stand up a parallel Microservices deployment, migrate parameter file logic across (most TABLE and MAP syntax transfers with minimal change, as noted in Chapter 3), validate it thoroughly against a non-production copy of the same data, and then cut over during a planned window once confidence is established, rather than attempting an in-place conversion of a live Classic installation.

18.3 GoldenGate for Data Warehouse Replication

Feeding a data warehouse or reporting platform is one of the oldest and most common GoldenGate use cases, and it has a few characteristics that differ from operational database-to-database replication. Warehouse targets often prefer bulk, batched loading patterns rather than row-by-row apply, so Replicat configurations feeding a warehouse frequently lean heavily on BATCHSQL and larger GROUPTRANSOPS values, and in some architectures Replicat writes to a staging table that a separate warehouse load process then merges on a schedule, rather than applying directly to the final reporting tables.

18.4 Real Time Analytics Feeds

Where genuine real time analytics is the goal rather than periodic batch refresh, the Kafka-based pattern described in Chapter 16 is generally preferred over a warehouse-style batched Replicat, since it decouples the rate of change capture from the rate of analytical consumption, allowing multiple independent analytics consumers to read the same stream of change events without adding load back onto the source database or onto Replicat itself.

18.5 Choosing Between Warehouse-Style and Streaming-Style Delivery

Consideration	Warehouse-Style Batched Replicat	Streaming (Kafka-Based) Delivery
Latency expectation	Minutes to hours is usually acceptable	Seconds or less is the typical expectation
Number of downstream consumers	Usually one warehouse or reporting platform	Often many independent consumers reading the same stream
Operational complexity	Familiar Replicat and staging table patterns	Requires Kafka cluster operation and handler configuration
Best fit	Traditional BI and reporting workloads	Event-driven applications and real time dashboards

Chapter 19: Quick Command Reference — GGSCI and Admin Client Side by Side

This chapter is meant to be used as a fast lookup during day-to-day administration, placing the Classic GGSCI command next to its Microservices Admin Client equivalent so that an engineer supporting both architectures does not need to hold two separate mental models while working.

19.1 Process Status and Control

```
-- Check status of all processes
GGSCI>       INFO ALL
adminclient> INFO EXTRACT *, DOMAIN admin
adminclient> INFO REPLICAT *, DOMAIN admin

-- Start / stop a process
GGSCI>       START EXTRACT EXT1
GGSCI>       STOP EXTRACT EXT1
adminclient> START EXTRACT EXT1, DOMAIN admin
adminclient> STOP EXTRACT EXT1, DOMAIN admin
```

19.2 Lag and Statistics

```
GGSCI>       LAG EXTRACT EXT1
GGSCI>       STATS REPLICAT REP1, TOTALSONLY *
adminclient> STATS EXTRACT EXT1, DOMAIN admin
adminclient> STATS REPLICAT REP1, DOMAIN admin
```

19.3 Adding Processes

```
GGSCI>       ADD EXTRACT EXT1, INTEGRATED TRANLOG, BEGIN NOW
GGSCI>       ADD EXTTRAIL ./dirdat/aa, EXTRACT EXT1
adminclient> ADD EXTRACT EXT1, INTEGRATED, TRANLOG, BEGIN NOW, DOMAIN admin
adminclient> ADD TRANDATA HR.EMPLOYEES, DOMAIN admin
```

19.4 Credential and Security Commands

```
GGSCI>       ADD CREDENTIALSTORE
GGSCI>       ALTER CREDENTIALSTORE ADD USER ggadmin, ALIAS gg_src
adminclient> ALTER CREDENTIALSTORE ADD USER ggadmin, ALIAS gg_src, DOMAIN admin
```

19.5 Checkpoint and Heartbeat Commands

```
GGSCI>       ADD CHECKPOINTTABLE HR.GGS_CHECKPOINT
GGSCI>       ADD HEARTBEATTABLE HR.GG_HEARTBEAT
adminclient> ADD CHECKPOINTTABLE HR.GGS_CHECKPOINT, DOMAIN admin
```

USING THIS CHAPTER

Keep this chapter bookmarked rather than memorized. The value of a quick reference is in how fast it can be checked mid-incident, not in reciting it from memory during an interview.

19.6 Final Word

Every chapter in this reference has tried to explain not just the command or parameter syntax, but the reasoning behind it — why Oracle built a feature a certain way, what production problem it solves, and what tends to go wrong when it is misconfigured. Commands and parameter syntax will keep evolving release after release, but the underlying engineering reasoning behind capture, trail, and apply changes far more slowly, and that reasoning is what actually carries an administrator through an unfamiliar version or an unexpected production incident.

Chapter 20: Glossary of Key Terms

This glossary is meant as a fast lookup for terms used throughout this reference, gathered in one place for convenience rather than requiring a search back through individual chapters.

Term	Meaning
Manager	Supervisor process in Classic Architecture that starts, restarts, and monitors Extract and Replicat, and manages trail purging and port allocation.
Service Manager	Microservices equivalent of Manager, capable of hosting multiple isolated deployments on one host.
Extract	Process that captures changes from the source database and writes them into a trail file.
Data Pump	A secondary Extract that forwards a local trail across the network, isolating capture from network interruptions.
Distribution Server	Microservices equivalent of a Data Pump, sending trail data to a Receiver Server.
Receiver Server	Microservices equivalent of a Collector, accepting incoming trail data on the target.
Replicat	Process that reads trail records on the target and applies them using SQL against the target database.

Term	Meaning
Trail File	Sequential binary file holding captured change records, self-describing through its trail header.
Checkpoint	Recorded position marking exactly how far a process has read or applied, enabling safe restart.
Checkpoint Table	Database table storing Replicat checkpoint information transactionally with applied data.
Integrated Capture	Capture mode where Extract requests change records from an in-database LogMiner-based process.
Integrated Replicat	Apply mode where an in-database inbound server manages apply parallelism automatically.
Coordinated Replicat	Apply mode using manually defined threads and hash-based row distribution.
Parallel Replicat	Apply mode using an automatically built transaction dependency graph for parallel apply.
Credential Store	Encrypted storage for database usernames and passwords, referenced by alias in parameter files.
Heartbeat Table	Mechanism for measuring true end-to-end replication lag using timestamped rows flowing through the pipeline.
Discard File	File capturing records Replicat could not apply, along with the associated error detail.
Logdump	Utility for reading trail files directly, independent of a live database connection.
Conflict Detection and Resolution (CDR)	Mechanism for detecting and automatically resolving conflicting changes in active-active topologies.
Loop Detection	Mechanism preventing a replicated change from being re-captured and sent back to its origin.
OCI GoldenGate	Oracle's fully managed cloud offering of the Microservices Architecture.
GoldenGate Veridata	Product for independently comparing source and target data to confirm replication correctness.